

介绍

原语雀文档链接: <https://yuque.antfin.com/amap-webx/sl2e5s/grx3wn86vqaa44zq>

简介

bff-vo 是为模板化架构提供服务模板数据的 faas 开发框架, 通过它我们可以做下面这两件事。

- 提供 Page 模板接口
- 提供 Vo 数据接口

类比说明

bff-vo 是生产模板化数据的, 现在我们将 bff-vo 类比为生产水果篮的工厂。
为了实现批量化生产, 整个工厂从架构上拆分为三个部门: 原料部、加工部、产品部。
工厂的运作流程是这样的:

- 产品部要生产面向孩子、年轻人、老年人三种类型的果篮
 - 产品
 - 孩子款: 青苹果2个、小橘子3个
 - 年轻人: 红苹果4个、大橘子5个
 - 老年人: 香蕉5个、大橘子2个
 - 职责: 基于水果品类进行二次打包为不同用户群体需要的产品
- 加工部需要提供青苹果、红苹果、小橘子、大橘子、香蕉五种类型的水果
 - 职责: 基于原料部提供的产地水果进行品类划分, 然后提供给产品部使用
- 原料部需要提供苹果、橘子、香蕉三种类型的水果
 - 职责: 将果园的水果完好无损的带回工厂, 给加工部使用

为什么要拆分三层？

- 职责清晰
 - 原料部：提供的是产地水果
 - 加工部：提供的是按品类划分后的水果
 - 从品类上划分，青苹果和红苹果是两个品，小橘子和大橘子是两个品
 - 产品部：提供的是面向用户的果篮
- 边界清晰
 - 原料部提供的是苹果，而不会是青苹果或者红苹果
 - 加工部提供的是青苹果和小橘子，不会是果篮
 - 产品部提供的是果篮，不会是青苹果
- 流程复用
 - 产品部生产了三种类型的果篮，但是他们复用了同一个加工部提供的品类水果
- 规模化生产
 - 规模化生产的必要条件是生产效率足够高
 - 以福特公司为例，它是这样实现汽车的规模化生产的
 - 采购100辆汽车的原材料。
 - 制造100辆汽车的所有零件（总共超过十万个）。
 - 组装100辆汽车，然后出售。
 - 接收顾客反馈，生产下一批次。
 - 该方式的特点是可以实现低成本生成大批量单一车型的汽车
 - 低成本
 - 车型单一
 - 与规模化生产相对应的是定制化生产
 - 采购一辆汽车需要的原材料、制造这辆汽车需要的零件、组件成整车、卖出。
 - 该方式的特点是生产效率低、成本高，但可以按照顾客的要求生产每一辆汽车，即定制化生产
 - 信息业务要实现 365 个行业的支持，必然要做规模化生产的路线
 - 即通过生产几种形态的产品（搜索=>分发=>详情=>交易）来满足对 365 个行业的支持

架构设计

职责边界

- 模板层 (Page)
 - 职责：输入的是聚合层数据，输出是模板化数据 (PMT 协议数据)
 - 边界：生成 PMT 协议数据
- 聚合层 (VO)
 - 职责：输入的是数据源数据，输出的是聚合后的模块数据
 - 边界：生成的是聚合模块，无视图协议（如不是 PMT 协议格式）
- 数据源 (DTO)
 - 职责：聚合商品、价格、店铺等数据，输出标准 DTO 数据，即数据源
 - 边界：生成的是数据源，数据源无法直接给组件消费，模块数据可直接给组件消费

Page 层

Page 层是编写 Page 接口的地方，它本质上是一个服务接口，提供数据给端上消费，与普通接口不同的是，Page 层有固定的职责以及返回的数据为模板数据 (PMT 协议数据)。

对 Page 接口有疑问可联系 [@继然](#)

信息前端新的 bff-vo 接口统一基于 Page 接口的形式编写，老的接口也会逐步升级。

Vo 层

Vo 层是编写 Vo 接口的地方，它与 Page 的区别是，它返回的是模块数据，且负责 DTO 到 VO 的转换。

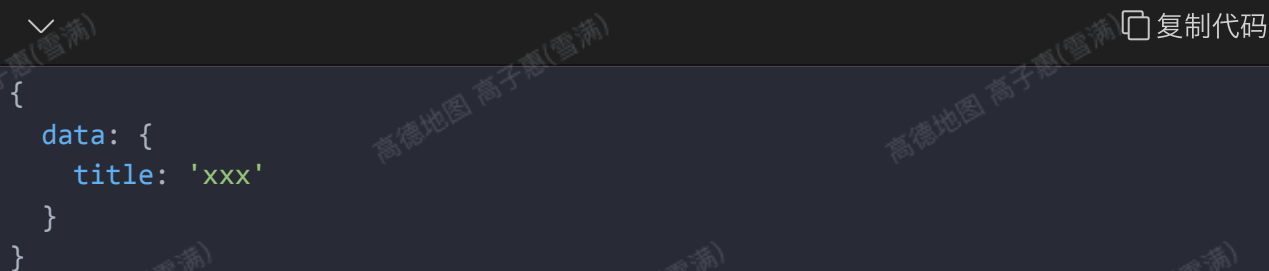
[vo 层开发](#)

模块和算子均编写在 amap_bundle_lib_vo 中。

[模块/算子开发](#)

模块

模块是一个可返回端上组件需要的数据的函数，它返回的数据格式如下：



```
{
  data: {
    title: 'xxx'
  }
}
```

复制代码

每一个模块可以绑定多个 card_id，至少会有一个 card_id 对应。

模块间可复用的代码，自行抽取为 utils 方法使用。